SOURCE-LEVEL GUIDE • APACHE/TVM-TIRX

Apache TVM TIRx 源码导读

TIRx 是为 agent 写 GPU kernel 而重建的编译器底座。这份导读从一个 add_one 出发，讲清它的 IR、TVMScript、lowering pipeline、CUDA 后端和跨语言扩展机制，以及贡献和定制时要知道的事。

apache/tvm-tirx @ 40d297a

怎么读这份导读

这份导读写给有编译器或 GPU 背景、第一次接触 TIRx 的工程师。读完你应该能做到：说清一段 TVMScript 怎么变成 CUDA 和 host 代码；知道新算子、新 pass、新后端该放在哪；判断 TIRx 适不适合做你的 kernel 工具链底座。

全文顺序：

1. **全景**：它要解决什么问题，要求和限制是什么，整体结构。
2. **IR 与对象系统**：节点、类型、不可变树、TVM-FFI 反射。
3. **TVMScript**：Python 函数怎么变成 IR。
4. **编译流水线**：`compile` 的每个阶段和每个 pass。
5. **符号分析**：`sym` 命名空间。
6. **CUDA 后端**：tile 操作、PTX 表、代码生成、运行时。
7. **扩展机制**：不重编编译器就能加类型、算子、pass、后端。
8. **端到端**：`add_one` 的完整旅程。
9. **许可证、贡献与定制、源码索引**。

> **要点 · 固定版本**
>
> `apache/tvm-tirx` @ `40d297a`（2026-11-05 提交），包版本 `0.1.0b1`。这是 beta 版本，文档和 API 还在快速变化。我没有构建或运行过这个项目，下面所有说法来自阅读源码和文档。

全景：问题、目标、限制

为什么还要一层编译器

README 的立场很直接：未来 GPU kernel 都会由 AI agent 开发、优化和验证（`README`）。那 agent 既然能直接写 CUDA、PTX 甚至 SASS，为什么还要编译器？

官方博客 给的回答分成两点：

需求	为什么 agent 自己做不好	TIRx 的回答
分析	逐字读程序能发现问题，但成本高。内存访问、同步这类检查用结构化算法更便宜	结构化 IR 加上符号分析、遍历工具
验证	程序对不对不能只靠 agent 的判断，需要有精确语义的程序表示	小而稳定的核心 IR
迭代速度	改算子要重编编译器，agent 写得越快，这部分占比越高	动态扩展：扩展放在独立仓库，运行时加载
语言选择	实验性分析适合 Python，高频 pass 适合 Rust/C++	Python、C++、Rust 一等支持
维护压力	kernel 可以生成了就丢，底层却要稳定	按稳定性分层：底座小而稳，上层随便变

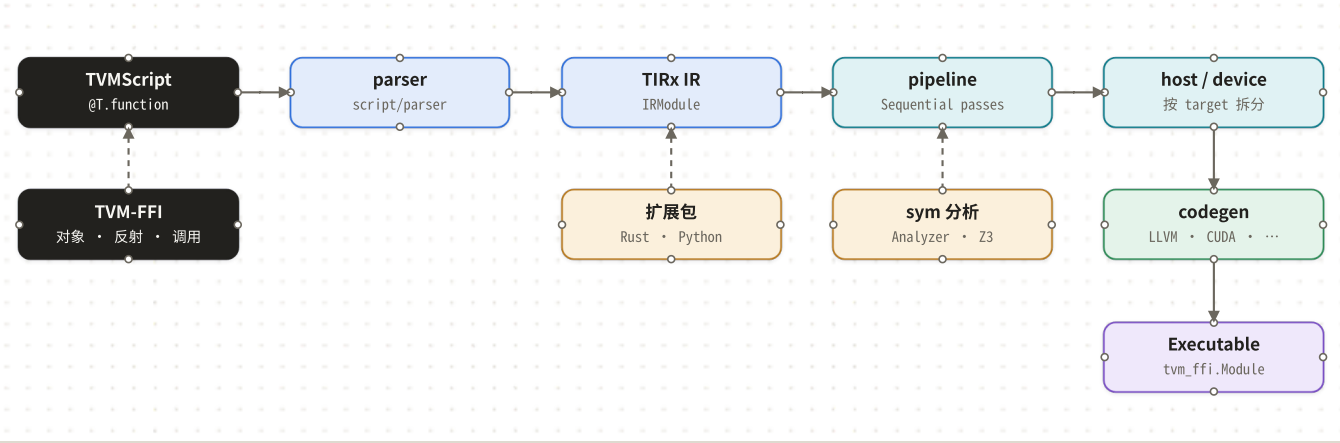
这和传统 DSL 的思路相反。编程总览 的说法是：成熟 kernel 可以把边界画得高，让编译器藏起线程分配和内存移动；但在硬件前沿，新指令先于编译器自动化出现，所以 TIRx 把边界**画得低**（`设计哲学`）。流水线结构、同步、角色分配、内存放置都直接写在源码里。

要求 / 目标 / 限制

项目	内容
核心目标	给 kernel 编译器和 agent harness 提供共享的 IR、分析、变换和代码生成底座
功能要求	TVMScript 写程序；核心 IR 打印/解析；host LLVM 和 CUDA 代码生成；多设备后端
规模压力	大程序上的 pass 速度，这是扩展推荐用 Rust 的原因（`文档`）
外部限制	依赖 TVM-FFI ABI；CUDA 代码最终交给 NVRTC 或 nvcc；LLVM 版本由 `mlc-llvm-static` 固定
关键手段	不可变 IR；开放的 `Op` 系统；反射；表驱动 PTX；target 决定 pipeline
许可证	Apache-2.0，属于 ASF 项目

总体结构

从 TVMScript 到可执行模块



深色是外部输入和依赖。TVM-FFI 是 `git 子模块`，提供对象和跨语言调用。分层说明见 `AGENTS.md 的架构概念`。

命名空间和目录的对应关系：

命名空间	职责	Python	C++
`ir`	共享的表达式、类型、语句、函数、模块	`ir/`	`src/ir`
`tirx`	张量程序构造、layout、分析、lowering	`tirx/`	`src/tirx`
`sym`	化简和符号推理	`sym/`	`src/sym`
`script`	解析和打印	`script/`	`src/script`
`backend`	目标相关的操作、变换、代码生成	`backend/`	`src/backend`
`runtime`	设备内存和模块加载	`runtime/`	`src/runtime`

它从哪来

TIRx 不是新写的，来自 `apache/tvm` 里的 TensorIR 一脉。LICENSE 的提交历史显示：2026-10-15 有一次 “Bringup tirx in standalone folder”，10-17 有 “Make standalone TIRx the repository root”（GitHub API 核实）。所以文档里还能看到 “Adapted from Apache TVM” 的标记（`示例`）。

IR 与对象系统

职责

IR 是一棵树：类型、表达式、语句、函数、模块。`IR 总览` 说明 IR 对象**不可变**：分析只读，变换构造新树，未改动的部分共享。

难点

这棵树要同时被 C++ pass、Python pass、Rust pass 操作，而且类型可以运行时新增。如果每种语言各有一套节点定义，动态扩展就做不到。

原理：一份对象，三种语言

所有节点都是 TVM-FFI 对象，用 `FooObj` 数据类加 `Foo` 引用包装，字段通过 `ffi::reflection::ObjectDef` 暴露（`C++ 约定`）。Python 用 `@tvm_ffi.register_object("type.Key")` 绑定同一个类型键（`Python 约定`）。因此 Python 看到的 `Add` 不是副本，而是 C++ 对象的句柄。

抽象	作用	位置
`Type`	值的类型：标量、张量	`type.h`
`Expr`	计算值	`expr.h`
`Stmt`	语句和顺序	`stmt.h`
`Op`	开放的算子注册表	`op.h`
`IRModule`	一组命名函数，加 attrs 和全局信息	`module.h`
`Pass`	模块到模块的变换	`transform.h`
`Layout` / `TileLayout`	逻辑张量到物理存储的映射	`layout.h`、`layout.py`
`Target`	编译目标和后端配置	`target.py`

核心节点少、算子开放是这里的关键设计。新增硬件指令时不用加节点，只要注册一个 `Op`，所以博客说底座能靠算子系统表示新的 CUDA/PTX，而不改 IR 基础设施。

看一个具体例子。`B[i] = A[i] + 1` 右边的值打印出来是：

```
Add
├── a: TensorLoad(source=A, indices=[i])
└── b: FloatImm(value=1.0)
```

来自 IR 总览，用 `ir.format_tree` 打印。

类比 · 带标准插座的墙

- **墙** = 核心 IR：节点集合小，不常改。
- **插座** = `Op` 注册表：什么设备都能插，形状统一。
- **电器** = 每条 PTX 指令、每个扩展算子。
- **电工手册** = 反射信息：Python、C++、Rust 都按同一份手册接线。

加一台新电器不用拆墙，这就是“不重建编译器就能扩展”的含义。

注意 · 容易误解的地方

`tvm_tirx.tirx` 和 `tvm_tirx.ir` 是两层。`ir` 放通用节点，`tirx` 在其上加张量语义。写 pass 前先确认节点属于哪一层，否则会在错误的 functor 上实现。

TVMScript：从 Python 到 IR

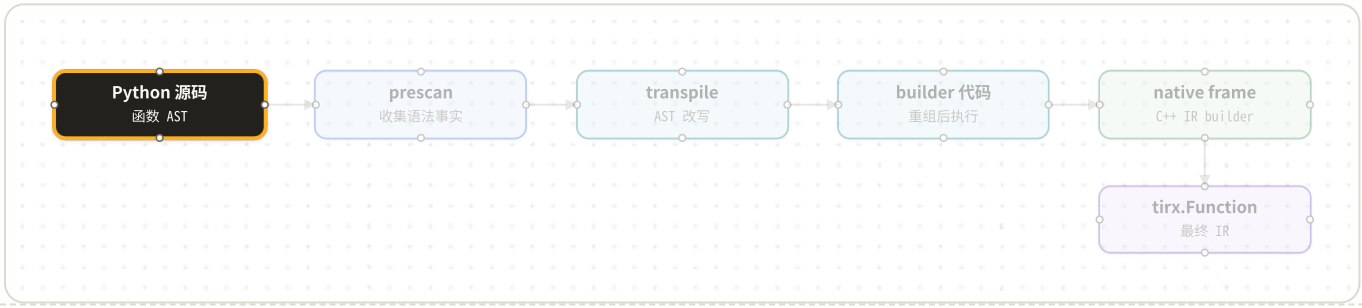
职责

TVMScript 是写 IR 的 Python 语法，也是 IR 打印出来的样子。这一点保证了打印和解析能往返（`打印与解析`）。

原理

`@T.function` **不执行函数体**。它拿到源码 AST，改写成调用 IR builder 的代码，再运行这段 builder。`transpile.py` 的文件头写得很清楚：入口先获取新的 AST，用 `PrescanContext` 收集语法事实，递归改写器消费这些事实；每个函数一个新的 `FunctionContext`。

▶ **@T.function 的执行过程** 1 / 5



1. 拿到源码

`make_decorator` 生成装饰器，`parse` 是解析入口。

2. 预扫描

`prescan.py` 先走一遍语法树，把事实记下来。这些事实之后不会被改写器修改。

3. 改写

`IRBuilderTranspiler` 是一个 `ast.NodeTransformer`，把 `for`、`if`、下标赋值改写成 builder 调用。

4. 重组并执行

`_recompose_builder` 把生成的 helper 组回来，入口执行它并释放临时捕获。

5. 进入 native

声明、参数、区域结果和最终 IR 都由 `C++ frame` 持有。Python 上下文只是临时翻译状态，不镜像构造状态。

Meta 编程

kernel 里大量重复的同步逻辑靠 `@T.inline` 和 `@T.meta_class` 复用（`metaprogramming`）。例子是 `pipeline.py`：`PipelineState`、`MBarrier`、`TMABar`、`Pipeline` 都在解析时发出 TIR，自身不会进入 IR。

`_tcgen05_commit_is_unicast` 的注释说明了为什么这段逻辑要留在 inline 函数之外：inline 求值器不短路条件表达式，这个判断需要真正的 Python 语义。这种细节在写自己的 meta 类时会遇到。

编译流水线

职责

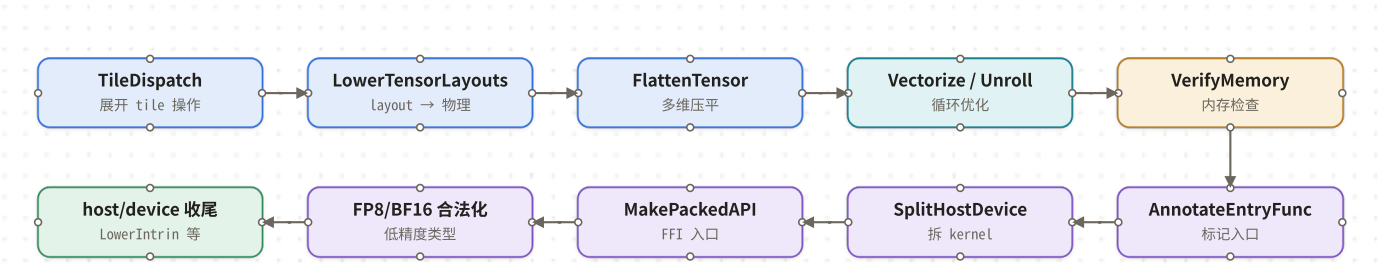
`compile` 把 `Function` 或 `IRModule` 变成 `Executable`。它做五步：确定绑定目标；确定搜索 pipeline 用的目标；确定 host 目标；运行 pipeline；拆分 host/device 后分别收尾和生成代码。

host 的默认选择有个细节：LLVM 启用就用 `llvm`，否则用 `c`（`_compile.py`）。

pipeline 的结构

pipeline 不是一个 pass 列表，而是**三元组**：lowering pass、host 收尾工厂、device 收尾工厂（`tirx_pipeline`）。

默认 pipeline 的阶段



第一排对应 `lowering 列表`，第二排对应 `finalize_tir_pipeline`。中间还有 `StmtSimplify`、`NarrowDataType`、`CommonSubexprElim`、`LowerThreadAllreduce`，图里省略了。

Pass	作用	实现
`TileDispatch`	把 tile 操作展开成线程级代码	`lower.cc`
`LowerTensorLayouts`	把 layout 声明变成地址计算	`lower_tensor_layouts.cc`
`FlattenTensor`	多维张量压成一维	`flatten_tensor.cc`
`VerifyMemory`	检查内存访问是否合法	`verify_memory.cc`
`SplitHostDevice`	把设备区域拆成 kernel，并生成 launch	`split_host_device.cc`
`MakePackedAPI`	生成 packed 调用约定的入口	`make_packed_api.cc`
`LowerTVMBuiltin`	host 侧运行时内建函数	`lower_tvm_builtin.cc`

调试和替换

- 设置 `TVM_TIRX_PRINT_AFTER_DISPATCH_OPS` 可以看 dispatch 之后的 IR（`代码`）。

- pass 配置项 `tirx.disable_vectorize`、`tirx.disable_cse_tir` 可以关闭对应优化。
- `register_tir_pipeline` 注册自定义 pipeline。工厂如果接受 `target` 关键字，就要自己负责绑定目标。

> **注意 · 容易误解的地方**
>
> `compile` 默认 pipeline 依赖 target，不是全局固定的。`Step 3` 走 `get_default_tir_pipeline`，后端可以提供自己的（如 `trn`）。另外 target 绑定发生在 lowering 线程 launch 之后，因为分类和 helper 特化要看完整的混合模块。

符号分析

职责

GPU kernel 里到处是形如 `(i * 256 + t) / n` 的索引。证明它们不越界、化简它们，是编译器的基本功。`sym` 把这部分独立出来，给 pass 和扩展直接用。

结构

`Analyzer` 是一个门面，组合多种分析器：

分析器	回答什么问题	位置
`ConstIntBoundAnalyzer`	这个表达式的常数上下界	`analyzer.h`
`ModularSetAnalyzer`	模余性质，如一定是 4 的倍数	`modular_set.cc`
`RewriteSimplifier`	基于规则的化简	`rewrite_simplify.cc`
`CanonicalSimplifier`	规范形化简	`analyzer.h`
`IntSetAnalyzer`	取值集合	`int_set.cc`
`Z3Prover`	交给 SMT 求解器	`z3_prover.cc`

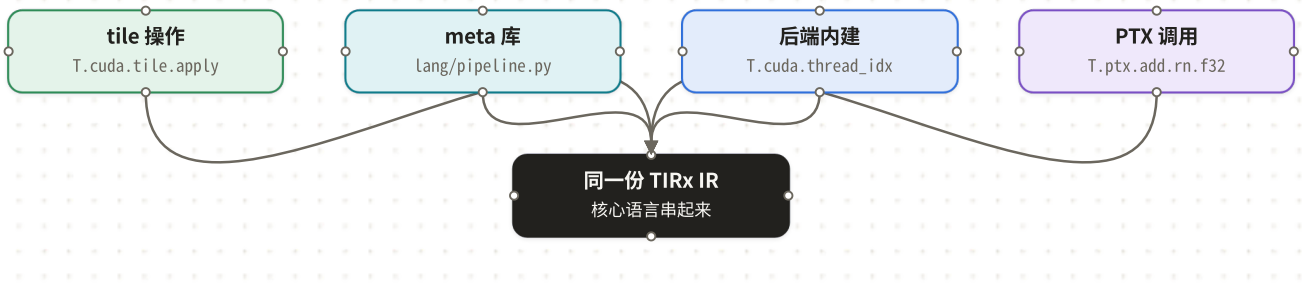
证明强度是可选的：`ProofStrength` 的默认档是先做默认分析，再用可选的 Z3 兜底。Z3 由构建选项控制，`TVM_TIRX_USE_Z3` 默认 `AUTO`。

`iter_affine_map.h` 处理迭代变量的仿射映射，配合 `IndexMap` 和 layout 使用。指南见 `符号整数分析`。

CUDA 后端

这是目前最完整的后端。写 kernel 时可以在下面这几层之间自由切换，同一个函数里也能混用。

同一个 kernel 里的四层写法



从左到右越来越接近硬件。核心语言提供循环、分支和张量访问，四层都落到同一份 IR 上（`Authoring Layers`）。

三个构造

编程模型靠三个构造组织（`Programming Model`）：

构造	含义
Execution scope	哪些线程参与、在什么层级：单线程、warp、整个 thread block
Tensor layout	逻辑张量怎么映射到物理存储
Tile operation dispatch	根据操作、scope、layout 和 target 选实现，并生成循环和寻址

表驱动的 PTX

`T.ptx.add.rn.f32(result, a, b)` 这种调用直接对应一条 PTX。`table.py` 有 1 万多行，是这部分的唯一事实来源：每个 `InstructionEntry` 记录操作数顺序和修饰符。

文件头定了几条规则：一个 entry 只对应一种语法形状；跨修饰符约束用纯函数 `check` 表达；谓词 `@p` 由框架统一处理；表本身不 import `tvm`，所以生成器可以独立加载它。它按 PTX ISA 9.4 编写，并用 CUDA 13.4 的 ptxas 核对。

这是表驱动的好处：新增指令形式只改表，桩代码、覆盖率和 helper 由 `gen_stubs`、`gen_coverage`、`gen_helpers` 生成。完整流程在 `PTX TableGen 指南`。

Tile 操作

TIRx 只提供两个 CUDA tile 操作（`tile 指南`）：

- `T.cuda.tile.apply`：重复一条指定的 PTX 指令，比如向量加载或矩阵乘。

- `T.cuda.tile.ewise`：对寄存器元素应用一个标量 TIRx 函数。

指令变体和同步由你自己选，编译器不替你决定。这和 CuTe DSL 的不同在于，tile 层是可选的，任何部分都能下探到原生指令。

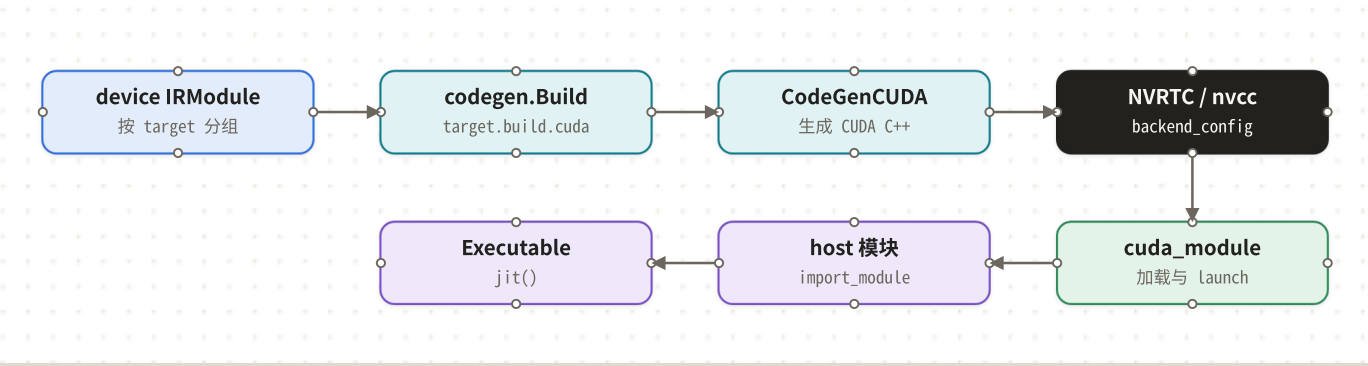
dispatch 实现在 `TileDispatcher`。它遇到 `device_entry` 区域时处理入口，然后展开 tile 调用。lowering 回调从 `DispatchContext` 取信息：target、thread scope、launch 参数、变量范围。回调可以请求私有分配，或在设备和 host 上插入初始化语句（`kPrivateAlloc`、`kDeviceInitStmt`、`kHostInitStmt`）。

同步：写成库

barrier 编排不在编译器里，而在 `lang/` 这个 Python 库里。`tile_scheduler.py`、`alloc_pool.py` 也一样。这体现了博客说的分层：会变的东西放在底座之上。

代码生成和运行时

CUDA 代码的去向



`codegen.Build` 按 `target.build.<kind>` 分发，CUDA 的注册在 `codegen_cuda.cc`。设备模块通过 `import_module` 挂到 host 模块下（`_codegen_host_and_device`）。

默认用 NVRTC，也可以切到 nvcc，配置在 `BackendConfig`。`Executable.jit` 解释了为什么需要它：返回的模块里可能还有待编译的 CUDA 源码和目标文件，第一次调用才完成编译和链接。

`cuda_fallback_module.cc` 这个名字暗示了没有 CUDA 运行时时的兜底。我只看了文件名，没有细读。

其他后端

`内置后端列表` 有 `cuda`、`metal`、`rocm`、`trn`、`opencl`、`vulkan`、`webgpu`、`hexagon`、`adreno`、`rust`。CUDA 之外的成熟度差别很大：`hexagon` 和 `metal` 在 C++ 侧只有 codegen 目录。AGENTS 明确提醒：测试被跳过只表示没覆盖，不代表后端可用（`原文`）。

扩展机制

这是 TIRx 区别于传统编译器的核心卖点。

四个扩展点

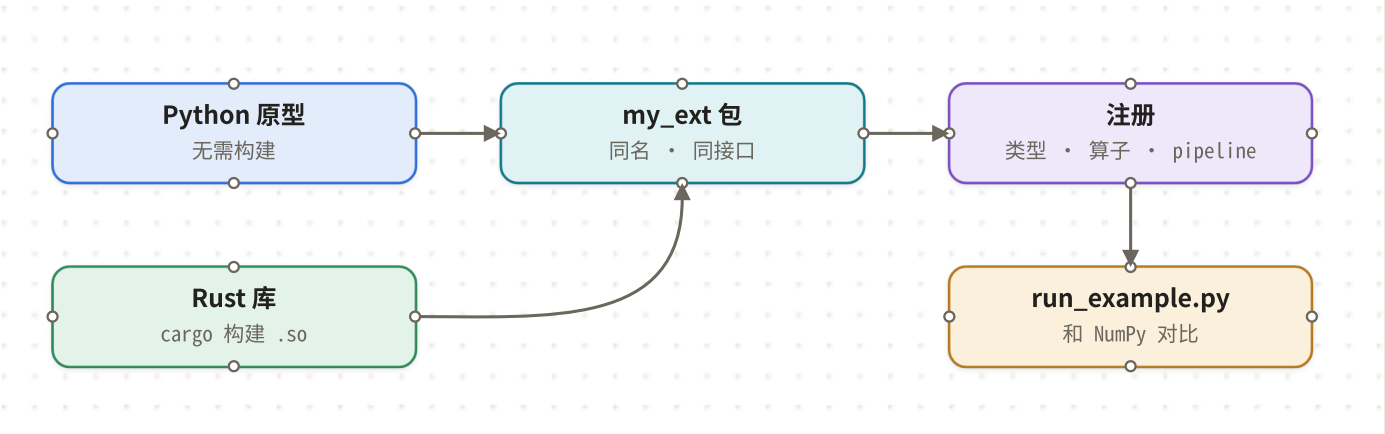
想加什么	怎么加	需要重编 TIRx 吗
新算子	注册 `Op`	不需要
新类型	定义 `TypeObj` 子类，走反射	不需要
新 pass	写 pass，放进自定义 pipeline	不需要
新后端	声明 `tvm_tirx.backends` entry point	不需要
新 PTX 形式	改 `table.py`	只改 Python
改核心节点	改 `include/tvm/ir/`	需要，而且应当避免

外部后端的入口写在 pyproject：`[project.entry-points."tvm_tirx.backends"]`，`import tvm_tirx` 时自动加载，可用 `TVM_TIRX_DEVICE_BACKEND_AUTOLOAD=0` 关闭（`_autoload_backends`）。

官方示例

`扩展示例` 实现同一个 `my_ext`：一个复数类型、几个算子、一个把它们降到标量运算的 pass。它写了两遍：

同一个扩展的两种实现



两个项目安装同一个分发包，所以示例程序不改一行就能切换。Python 版本见 `python/`，Rust 版本见 `lib.rs`。推荐路径是先用 Python 验证想法，再搬到 Rust。

Rust 这边，`rust/` 是独立 crate，IR 绑定由 `tvm-ffi-stubgen` 在标记之间生成，所以不要手改那些区段（`说明`）。`import tvm_tirx` 时还会加载 `libtvm_tirx_rust`，注册静态初始化块里的算子和类型，所以**改了这些要重新跑 editable install**。

类比 · 先搭木模，再做铸件

- **木模** = Python 版扩展：改起来快，大程序上慢。
- **铸件** = Rust 版扩展：要构建，但快且内存安全。
- **模具尺寸** = 反射接口：两边一样，所以替换时上下游都不用动。

agent 先用木模试形状，确定了再铸造。这就是文档推荐的路径，也是博客说的“不把全局语言选择压到局部决策上”。

端到端：add_one 的一生

下面用 `add_one_cuda.py` 走一遍。

```
n = T.dynamic("n")

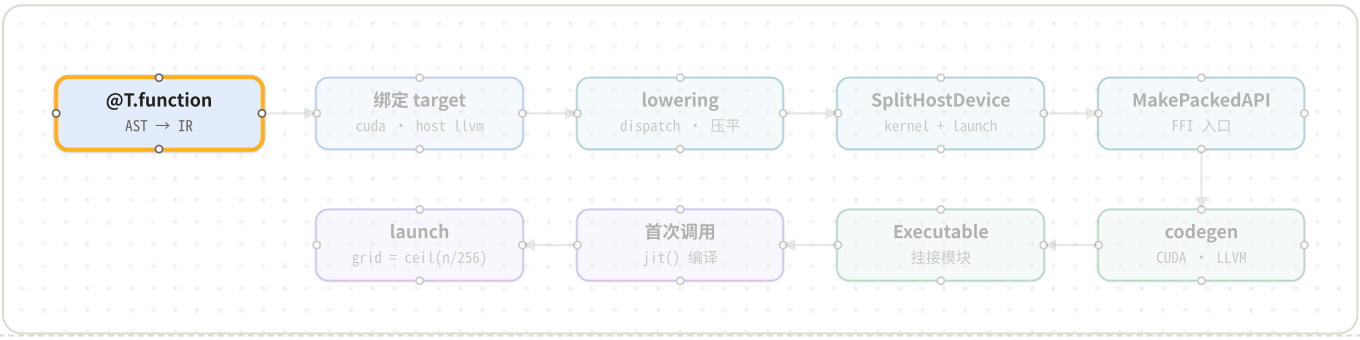
@T.function
def add_one(A: T.Tensor((n,), T.float32), B: T.Tensor((n,), T.float32)):
    T.device_entry(launch=T.cuda.LaunchConfig(grid=T.ceildiv(n, 256), block=256))
    i = T.cuda.block_idx("x") * 256 + T.cuda.thread_idx("x")
    if i < n:
        B[i] = A[i] + 1

add_one_compiled = tvm_tirx.compile(add_one, target="cuda")
```

简化自 `add_one_cuda.py`。

这里没有 parallel-for。线程索引是自己算的，grid 也是自己写的。这就是“边界画得低”的具体样子。

▶ add_one 从源码到 GPU 1 / 9



1. 解析

`@T.function` 改写 AST，得到一个 `tirx.Function`。`T.device_entry` 变成一个区域语句。

2. 绑定目标

`compile` 把函数包成 `IRModule`，确定 target 和 host。`target="cuda"` 时架构可以从本机探测，离线构建要写明架构（`compiling 指南`）。

3. lowering

默认 pipeline 跑 dispatch、layout、flatten、窄化、向量化、化简和 `VerifyMemory`。

4. 拆分

`SplitHostDevice` 把设备区域抽成 kernel，并在 host 一侧生成 launch。这里用 `DeviceInfoCollector` 收集设备侧信息。

5. ABI

`MakePackedAPI` 生成 packed 入口，这样 PyTorch 张量能直接传进来（通过 DLPack，由 TVM-FFI 完成）。

6. 代码生成

host 走 LLVM，device 由 `CodeGenCUDA` 生成 CUDA C++。

7. 组装

设备模块 `import_module` 到 host 模块，包成 `Executable`。

8. 第一次调用

`add_one_compiled(a, b)` 触发 `jit()`，完成编译和链接，然后调用 `main`。

9. 运行

运行时模块 launch kernel，grid 是 `ceil(n / 256)`，每个 block 256 个线程。`export_library` 可以把结果存成 `.so`，用 `load_add_one_cuda.py` 重新加载，甚至从 `C++` 加载。

步骤里“通过 DLPack”一句是我根据 TVM-FFI 的通用机制推断的，没有在这个调用路径上逐行核实。

开源许可证

部分	许可证	证据
TIRx 主体	Apache-2.0	`LICENSE`、`pyproject`
`builtin_fp16` 相关代码	见单独许可证文本	`licenses/`
TVM-FFI 子模块	由 `apache/tvm-ffi` 自己声明	`.gitmodules`

Apache-2.0 允许商用、修改和闭源分发，要保留版权声明和 NOTICE，修改过的文件要标明，含专利授权。

场景	允许吗	要做什么
公司内部使用	允许	无
给上游提 PR	允许	ASF 项目通常要求较大贡献者签 ICLA，请看 ASF 官方说明
公开 fork	允许	保留 LICENSE、NOTICE；不要用 Apache 商标暗示官方
发安装包	允许	带上 `NOTICE` 和第三方许可证
对外提供服务	允许	Apache-2.0 没有网络条款
闭源扩展	允许	动态扩展在自己的包里即可

历史：LICENSE 文件在 2026-10-15、10-17 随独立化重构进入当前仓库，更早的提交来自 `apache/tvm`（GitHub API 核实，我没有逐条 diff）。“Apache”和“TVM”是 ASF 商标，fork 命名要注意。ICLA 那一条来自 ASF 的一般做法，不是这个仓库里读到的。

> **不是法律意见**
>
> 这是工程师读代码时的理解，真要分发请咨询律师。

贡献与定制

开发环境

```
git clone --recursive https://github.com/apache/tvm-tirx
cd tvm-tirx
uv sync --no-default-groups --group test
uv run --no-sync pytest -v tests/python/ir/test_type.py
uv run --no-sync pre-commit run --all-files
```

命令来自 `AGENTS.md`。要求 Python 3.10+ 和 C++20 编译器。改了 C++ 要重新跑 editable install，纯 Python 改动立即生效。

项目规则

- 提交信息用标签风格：`[FIX]`、`[REFACTOR]`、`[FEAT]`、`[DOCS]`、`[CI]`（`规则`）。
- C++：`.cc` / `.h`，Google 风格，100 列；保留 Apache 许可证头。
- Python 导出、反射的 C++ API 和参考文档要保持一致。
- 改 CUDA launch 时，pre-commit 还会检查生成文件（`generate.py`）。
- CI 分两部分：GitHub Actions 跑 macOS、Windows 和 lint；Jenkins 跑 ARM、GPU、EXTRA（`jenkinsfile`）。

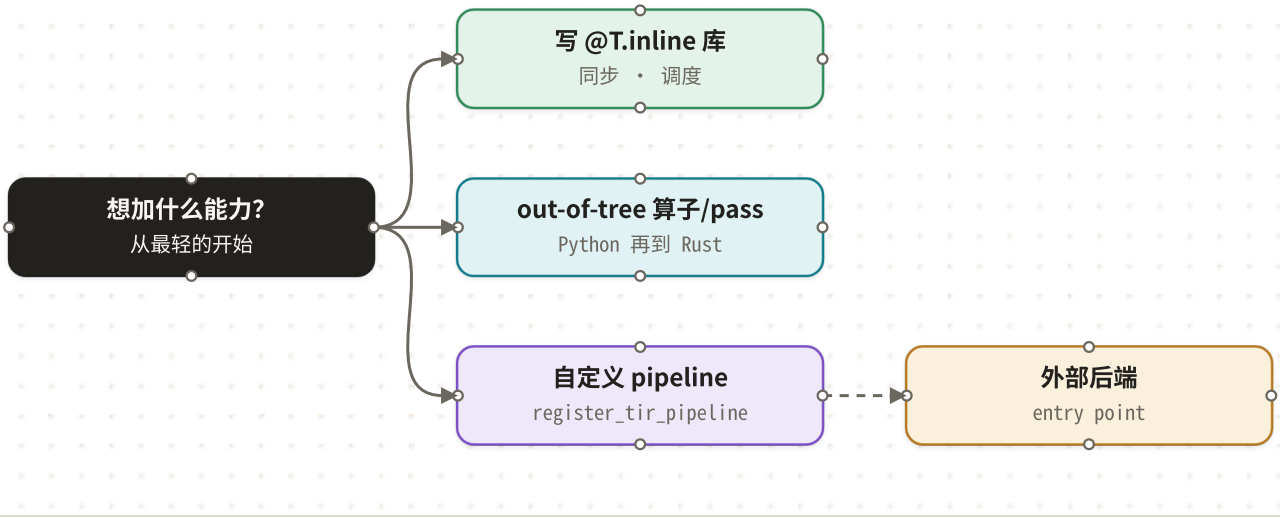
适合入手的方向

- **PTX 表覆盖**：对照 ISA 补指令形式，范围小，验证清楚。
- **文档示例**：`docs` 里的示例都由 `examples` 支撑，补充容易验证。
- **Rust 绑定**：博客说 Rust 还在进行中。
- **其他后端的测试覆盖**。

需要避开：给核心 IR 加节点。项目的核心取舍就是保持核心小，这类 PR 大概率要长时间讨论（这是我根据设计文档的推断）。

定制路线

我想在 TIRx 上做点什么



从上到下越来越重。绝大多数需求不需要 fork。

路线	代价	依赖	适合
meta 库	低	Python	复用同步和调度模式
扩展包	中	TVM-FFI 反射	新类型、算子、分析
自定义 pipeline	中	理解 pass 顺序	改变优化流程
外部后端	高	codegen 和运行时	新硬件
fork 核心	很高	长期跟随上游	尽量避免

仓库里没有的东西

- **agent harness 本身**：TIRx 是底座，不含 agent 循环和 cost model。博客提到的 TIRx Harness 是另一个项目。
- **TVM-FFI 实现**：在子模块里。
- **GPU kernel 库**：这里没有现成的生产 kernel。
- **形式化语义**：博客把验证作为方向，这个仓库里我没有看到 Lean 之类的形式化定义。

附录：源码索引

主题	链接
开发指南	`AGENTS.md`
构建	`CMakeLists.txt`、`pyproject.toml`
编译入口	`_compile.py`
pipeline	`compilation_pipeline.py`
解析器	`script/parser`
pass 实现	`src/tirx/transform`
分析	`src/tirx/analysis`
符号分析	`analyzer.h`
tile lowering	`lower.cc`
PTX 表	`table.py`
CUDA 代码生成	`codegen_cuda.cc`
CUDA 运行时	`cuda_module.cc`
Target 标签	`target_tags.py`
快速开始	`quick_start.rst`
扩展教程	`defining_ir_extensions.rst`
Rust	`lib.rs`

外部资料：官方文档、设计博客。